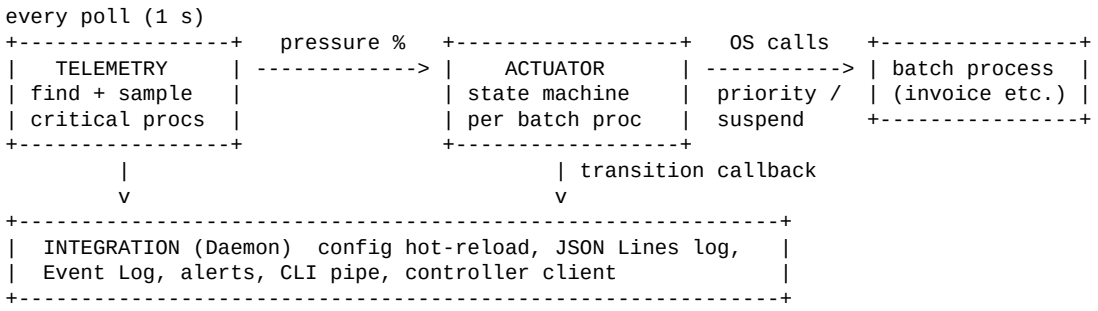
Configuration, Logging & Integration Module

Module owner: **Pranav** | Logistics server resource monitor & priority throttler (Windows, C++17)

1. What this module does

This module **connects and operates** the system. It loads and validates *config.json* (and re-applies it live when the file changes), writes every decision to a rotating JSON Lines log and the Windows Event Log, and runs the integration loop (*Daemon*) that wires Telemetry to the Actuator once per second. Around that loop it provides the local control pipe used by the CLI, config tamper protection, suspension alerts, the client and the central controller for multi-server coordination, the installer / uninstaller and the log-viewer graphs.

2. Where it fits in the system



Thresholds (defaults, all configurable): throttle at 85% for 3 samples, suspend at 90% for 5 samples after throttling, recover below 60% for 5 samples, 1 sample per second.

3. Files in this module

File	Purpose
Source/Config/Config.h/.cpp, config.json	DaemonConfig struct, ParseConfig / LoadConfigFile with field-by-field validation, sample config.
Source/Logging/JsonLinesLogger.h/.cpp	Thread-safe JSON Lines writer with size-based rotation; LogSample, LogThresholdEvent, LogActuatorAction, LogEvent.
Source/Logging/EventLogWriter.h/.cpp	Writes actuator actions to the Windows Application log (IDs 1000/1001/1002).
Source/Daemon/Daemon.h/.cpp	The integration loop: poll, pressure, adaptive thresholds, actuator, hot-reload, commands, status.
Source/Daemon/ControlPipe.h/.cpp	Named-pipe server/client for the CLI (admin-only DACL).
Source/Daemon/ConfigSecurity.h/.cpp	ACL check (is config writable by non-admins?) and SHA-256 of the file.
Source/Daemon/Alerts.h/.cpp	Popup (WTSSendMessage) and webhook (WinHTTP POST) on suspension.
Source/Daemon/Coordination.h/.cpp	Agent side of multi-server coordination (TCP, JSON lines, reconnect with back-off).
Source/Controller/controller.cpp	ThreadHaulController: accepts agents, logs, computes cluster bias, operator commands.
Source/Scripts/install.ps1, uninstall.ps1, log_viewer.py; build.cmd	Service installation, removal, graphs, one-step build.
Source/Tests/ConfigLoggingTests.cpp	91 unit checks for validation, optional sections, log schema and rotation.

4. How one poll and one config reload work

Every poll (default 1 s): check config.json timestamp/size -> snapshot processes -> Telemetry samples critical processes -> log samples and lifecycle events -> pressure = worst signal -> pick up new batch processes -> adaptive thresholds (+ controller bias) -> threshold_event lines -> ActuatorManager::UpdateAll(pressure) -> each transition is logged to JSON Lines and the Event Log, sent to the controller, and alerts fire on suspension -> self_monitor line every 30 s.

Config reload: file changed -> parse + validate the whole file -> ACL check -> if valid, ApplyConfig() and log config_reload success with the new SHA-256; if invalid, log the error and keep the previous config.

5. Windows APIs and libraries used

API / library	Where	What it is used for
nlohmann::json (third_party/nlohmann/json.hpp)	Config.cpp, JsonLinesLogger.cpp	Parses config.json and builds every log line (ordered_json keeps log_type and timestamp first).
MultiByteToWideChar / WideCharToMultiByte (CP_UTF8)	Config.cpp, logger	UTF-8 config text to wide process names and back for logs.
CreateFileW(FILE_APPEND_DATA) / WriteFile / MoveFileExW / DeleteFileW	JsonLinesLogger.cpp	Append log lines; rotate threadhaul.jsonl -> .1 -> .2 ... and delete the oldest.
GetSystemTime	JsonLinesLogger.cpp	ISO 8601 UTC timestamps with milliseconds.
RegisterEventSourceW / ReportEventW / DeregisterEventSource	EventLogWriter.cpp, main.cpp	Windows Event Log entries (information or warning; error 1900 on a startup failure).
GetFileAttributesExW	Daemon.cpp	Last-write time + size of config.json each poll: change detection for hot-reload.
GetNamedSecurityInfoW + GetAce + CreateWellKnownSid + EqualSid	ConfigSecurity.cpp	Reads the config file's ACL and checks whether Everyone / Users / Authenticated Users / Interactive have write rights.
BCryptOpenAlgorithmProvider(SHA256) / BCryptHashData / BCryptFinishHash	ConfigSecurity.cpp	SHA-256 of config.json logged on start and every reload (tamper evidence).
CreateNamedPipeW, ConnectNamedPipe, ConvertStringSecurityDescriptorToSecurityDescriptorW	ControlPipe.cpp	\\.\pipe\ThreadHaulDaemon with an SDDL DACL that admits only SYSTEM, Administrators and the daemon's account; PIPE_REJECT_REMOTE_CLIENTS.
WaitNamedPipeW, CreateFileW, SetNamedPipeHandleState	ControlPipe.cpp	CLI client side.
WTSGetActiveConsoleSessionId + WTSSendMessageW	Alerts.cpp	Popup in the logged-in user's session, which works from a Session 0 service.
WinHttpOpen / WinHttpConnect / WinHttpSendRequest	Alerts.cpp	Webhook POST (Teams, Slack, email relay).
WSAStartup, GetAddrInfoW, socket/connect/send/recv/select	Coordination.cpp, controller.cpp	TCP link between each daemon and the controller.
sc.exe create / failure / privs / sidtype, icacls, New-EventLog	install.ps1	Service registration, restart-on-failure, least privileges, config ACLs, event source.

6. Main functions written in this module

Function	What it does
config::ParseConfig(text) / LoadConfigFile(path)	Validates every field (type, range, cross-field rules such as low < high <= critical, no name both critical and batch) and throws ConfigError naming the field.
JsonLinesLogger::LogActuatorAction(...)	Writes the actuator_action line: batch and critical PIDs, action, from/to state, CPU%, actuation_succeeded, trigger.
JsonLinesLogger::LogEvent(type, fields)	Generic writer for all other log kinds (sample, lifecycle, config_reload, self_monitor, alert ...).
JsonLinesLogger::Rotate()	Size-based rotation keeping max_rotated_files old files.
Daemon::Initialize()	Loads and verifies config, creates the ActuatorManager with the logging callback, starts the control pipe, logs service start.
Daemon::Run(stopEvent)	Every interval: CheckConfigReload() then Poll(); on stop: restore every process, log, then disconnect from the controller.
Daemon::Poll()	Telemetry -> lifecycle logs -> pressure -> batch discovery -> adaptive thresholds -> threshold events -> ActuatorManager::UpdateAll -> self-monitor.
Daemon::ApplyConfig(cfg)	Applies a new config live: logger, thresholds, tracked names, protection list, alerts, coordination; restores batch processes whose name was removed.

Function	What it does
Daemon::OnTransition(...)	Callback for every Actuator transition: JSON log, Event Log, controller event, and alert if suspended.
Daemon::HandleCommand(line)	Executes CLI / controller commands (throttle, suspend, resume, release, status, stop) and returns JSON.
IsWritableByNonAdmins(path) / Sha256File(path)	Config tamper protection; with security.require_protected_config the daemon refuses an unprotected file.
CoordinationClient::Session()	hello (with shared token) -> heartbeats -> queued events -> receive bias and commands.
Controller Rebalance()	If >= cluster-percent of agents are above --pressure, send every agent bias = --bias; clear it when the cluster calms down.

7. Robustness and safety handled here

• **Invalid config at startup** stops the service with a message naming the field (console stderr, or Event Log ID 1900 for the service).

• **Invalid config during hot-reload** is rejected and logged; the daemon keeps running on the last good config.

• **Config tamper protection:** installer ACL (SYSTEM + Administrators write, service read, Users nothing), runtime ACL check and SHA-256 logging.

• **Least privilege:** the service keeps only SeDebugPrivilege and SeChangeNotifyPrivilege (sc privs); a LocalService option exists.

• **Control pipe security:** local only, DACL restricted to administrators, SYSTEM and the daemon's own account.

• **Controller down:** the daemon keeps protecting the server locally and reconnects with 1-30 s back-off; the bias is reset to 0 while disconnected.

8. How to demonstrate it

• `ThreadHaulDaemon.exe validate-config Source\Config\config.json`, then break a value and run it again to see the field-specific error.

• Hot-reload (RUNNING.md 4.5): a valid edit is applied, an invalid one is rejected and logged.

• Graphs: `python Source\Scripts\log_viewer.py demo.jsonl --out demo.png`.

• Service: `install.ps1` as Administrator, then Event Viewer -> Application -> source ThreadHaulDaemon.

• Multi-server (RUNNING.md 6): ThreadHaulController shows [connect], [event] and [cluster] lines; type `list`.

9. Likely review questions

Q. Why JSON Lines rather than one JSON document?

A. Each line is a complete record, so the file can be appended to, rotated and tailed safely, and a crash never corrupts earlier entries.

Q. Why both a log file and the Event Log?

A. The file is detailed and machine-readable for graphs; the Event Log is where Windows administrators and monitoring tools already look.

Q. How does hot-reload avoid partial reads?

A. The whole file is parsed and validated before anything is applied; a half-written or invalid file is simply rejected and retried on the next change.

Q. Why does the controller send a bias rather than commands?

A. Each server still decides locally from its own measurements; the bias only makes cooler servers start shedding batch load earlier during a fleet-wide peak, so a controller outage never stops local protection.

See also: README.md (overview), RUNNING.md (how to run), INTERFACES.md (cross-module contract), Source/Docs/literature-review.md (design rationale).